

Guardian Escolar - Funcionamiento de la app web Angular

Este documento explica el funcionamiento actual de la aplicacion web de **Guardian Escolar**, desarrollada con **Angular**. Esta pensado como guia de repaso: para entender que hace la web, para que sirve cada parte, cuales son sus dependencias, como esta organizada y que codigo es importante reconocer.

Proyecto web original:

```
dvlp-front/dvlp-web
```

Este documento fue creado junto al documento de MAUI para tener ambos repasos en un mismo lugar.

1. Objetivo de la app web

La app web de Guardian Escolar funciona como plataforma administrativa del sistema. Su objetivo es permitir que usuarios con roles administrativos puedan gestionar informacion relacionada con colegios, administradores, estudiantes, acudientes, conductores, familias, buses, paradas y rutas.

Tambien tiene una parte publica para presentar el producto y permitir el acceso al login.

En el estado actual, la web cuenta con:

- Pagina publica de inicio.
- Pagina de contacto.
- Login con formulario reactivo.
- Conexion de login al backend.
- Guardado de token y datos del usuario en `localStorage`.
- Interceptor HTTP para enviar el token en las peticiones.
- Dashboard de administrador.
- Dashboard de superadministrador.
- Vistas de gestion para usuarios, transporte, colegios y administradores.
- Componentes reutilizables de registro, listado, modales y navegacion.
- Flujos para recuperar contrasena.
- Flujos para cambiar correo, contrasena y telefono.
- Modal de idioma.
- Modal de tema.
- Traducciones con `@ngx-translate`.
- Configuracion para SSR/hydration.

Importante: varias pantallas ya tienen interfaz y comportamiento visual, pero muchos servicios CRUD todavia estan vacios o trabajan con datos de prueba.

2. Tecnologia usada

La aplicacion usa **Angular 21** con componentes standalone.

Archivo principal de dependencias:

```
dvlp-front/dvlp-web/package.json
```

Scripts principales:

```
{
  "start": "ng serve",
  "build": "ng build",
  "watch": "ng build --watch --configuration development",
  "test": "ng test",
  "serve:ssr:dvlp-web": "node dist/dvlp-web/server/server.mjs"
}
```

Esto significa:

- `npm start` levanta la app en modo desarrollo.
- `npm run build` compila la app para produccion.
- `npm test` ejecuta pruebas.
- `serve:ssr:dvlp-web` sirve la version SSR compilada.

3. Dependencias principales

Dependencias importantes del proyecto:

```
"@angular/core": "^21.2.9",
"@angular/router": "^21.2.9",
"@angular/forms": "^21.2.9",
"@angular/common": "^21.2.9",
"@angular/material": "^21.2.3",
"@angular/cdk": "^21.2.3",
"@angular/ssr": "^21.2.2",
"@ngx-translate/core": "^17.0.0",
"@ngx-translate/http-loader": "^17.0.0",
"@phosphor-icons/web": "^2.1.2",
"express": "^5.1.0",
"rxjs": "~7.8.0"
```

Para que sirve cada una:

- `@angular/core`: base de Angular.
- `@angular/router`: manejo de rutas y navegacion.
- `@angular/forms`: formularios reactivos y formularios con `ngModel`.

ngModel es una herramienta del marco de trabajo Angular. Conecta una casilla de texto en tu pantalla con un dato en tu código. Esta conexión funciona en dos vías. Si el usuario escribe en la pantalla, el dato cambia en tu código. Si tu código cambia el dato, la pantalla se actualiza sola.

- `@angular/material`: componentes visuales e iconos Material.
- `@angular/cdk`: utilidades base para componentes Angular.
- `@angular/ssr`: soporte para renderizado del lado del servidor.
- `@ngx-translate/core`: sistema de traducciones.
- `@ngx-translate/http-loader`: carga archivos JSON de idioma.
- `@phosphor-icons/web`: librería de iconos.
- `express`: servidor usado por SSR.
- `rxjs`: manejo de observables, peticiones y flujos asincronos.

4. Archivos principales de arranque

Archivo	Para que sirve
<code>src/main.ts</code>	Punto de entrada del navegador. Arranca Angular.
<code>src/main.server.ts</code>	Punto de entrada para SSR.
<code>src/server.ts</code>	Servidor Express para SSR.
<code>src/app/app.config.ts</code>	Configuración global de providers.
<code>src/app/app.routes.ts</code>	Rutas principales de la aplicación.
<code>src/app/app.html</code>	Plantilla raíz de la app.
<code>src/styles.css</code>	Estilos globales.
<code>src/material-theme.scss</code>	Tema de Angular Material.
<code>angular.json</code>	Configuración general de Angular CLI.

SSR Server-Side Rendering (en español: Renderizado en el Lado del Servidor) Es una técnica de desarrollo web donde el servidor genera el HTML completo de una página con todos sus datos listos, y se lo envía al navegador ya armado, en lugar de dejar que el navegador lo construya desde cero.

5. Configuración principal de Angular

Archivo:

```
src/app/app.config.ts
```

Código importante:

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideBrowserGlobalErrorListeners(),
```

```
provideRouter(routes),
provideHttpClient(withFetch(), withInterceptors([authInterceptor])),
provideClientHydration(withEventReplay()),
provideTranslateService({
  loader: provideTranslateHttpLoader({
    prefix: '/assets/i18n/',
    suffix: '.json',
  }),
}),
],
};
```

Esto configura:

- Rutas principales con `provideRouter(routes)`.
- Cliente HTTP con `provideHttpClient`.
- Interceptor de autenticacion con `withInterceptors([authInterceptor])`.
- Hydration para SSR con `provideClientHydration`.
- Traducciones cargadas desde `/assets/i18n/*.json`.

6. Variables de entorno

Archivos:

```
src/environments/environment.ts
src/environments/environment.development.ts
```

Configuracion actual:

```
export const environment = {
  apiUrl: 'http://localhost:8080'
};
```

Esto significa que la web espera comunicarse con el backend en:

```
http://localhost:8080
```

El servicio de autenticacion usa esa URL para llamar:

```
http://localhost:8080/api/login
```

7. Rutas principales

Archivo:

```
src/app/app.routes.ts
```

Rutas publicas:

```
{ path: '', redirectTo: 'home', pathMatch: 'full' },  
{ path: 'home', component: Home },  
{ path: 'contact', component: Contact },
```

Rutas de autenticacion:

```
{  
  path: 'auth',  
  children: [  
    { path: 'login', component: Login },  
    { path: 'forgot-password', component: ForgotPassword, children:  
      forgotPasswordRoutes }  
  ]  
}
```

Dashboards:

```
{ path: 'dashboard-admin', component: DashboardAdmin },  
{ path: 'dashboard-superadmin', component: DashboardSuperadmin },
```

Rutas del administrador:

```
{ path: 'admin/usuarios', component: Estudiantes },  
{ path: 'admin/padres', component: Padres },  
{ path: 'admin/conductores', component: Conductores },  
{ path: 'admin/familias', component: Familia },  
{ path: 'admin/buses', component: Buses },  
{ path: 'admin/paradas', component: Paradas },  
{ path: 'admin/rutas', component: Rutas },  
{ path: 'admin/informacion', component: InformationAdmin },
```

Rutas de cambios de perfil:

```
{ path: 'admin/change-email', component: ChangeEmail, children: changeEmailRoutes  
},  
{ path: 'admin/change-password', component: ChangePassword, children:
```

```
changePasswordRoutes },
{ path: 'admin/change-contact', component: ChangeContact, children:
changeContactRoutes },
```

Rutas del superadministrador:

```
{ path: 'superadmin/admins', component: Admins },
{ path: 'superadmin/schools', component: Schools },
```

Ruta de respaldo:

```
{ path: '**', redirectTo: 'home' }
```

Si una ruta no existe, Angular redirige a **home**.

8. Parte publica

Ruta principal:

```
/home
```

Archivos:

```
src/app/features/public/home/home.ts
src/app/features/public/home/home.html
```

La pagina de inicio carga:

```
<app-navbar></app-navbar>
<app-main></app-main>
<app-footer></app-footer>
```

Para que sirve:

- **Navbar**: barra superior publica.
- **Main**: seccion principal tipo landing con informacion del sistema.
- **Footer**: pie de pagina.

La pagina publica presenta Guardian Escolar como sistema de monitoreo GPS, alertas, reportes y control de rutas escolares.

9. Contacto

Ruta:

```
/contact
```

Archivos:

```
src/app/features/public/contact/contact.ts  
src/app/features/public/contact/contact.html
```

Para que sirve:

- Mostrar canales de contacto.
- Presentar correo, telefono, WhatsApp y redes.
- Permitir que un visitante se comunique con el equipo.

Los textos se toman de las claves `contact` en los JSON de traduccion.

10. Login

Ruta:

```
/auth/login
```

Archivos:

```
src/app/features/public/auth/login/login.ts  
src/app/features/public/auth/login/login.html  
src/app/core/services/auth.service.ts
```

El login usa formularios reactivos:

```
this.form = this.fb.group({  
  email: ['', [Validators.required, Validators.pattern(emailPattern)]],  
  password: ['', [Validators.required]]  
})
```

Validaciones:

- El correo es obligatorio.
- El correo debe cumplir un patron.

- La contraseña es obligatoria.

Al enviar:

```
login() {  
  if (this.form.invalid) {  
    this.form.markAllAsTouched();  
    return;  
  }  
  
  const { email, password } = this.form.value;  
  this.authService.login(email, password).subscribe({  
    next: (res) => {  
      const roles = res.roles;  
      if (roles.includes('admin')){  
        this.router.navigate(['/dashboard-admin'])  
      } else {  
        this.router.navigate(['/home'])  
      }  
    },  
    error: () => alert('Credenciales invalidas')  
  })  
}
```

Funcionamiento:

- Si el formulario es invalido, marca todos los campos como tocados.
- Si es valido, llama a `AuthService.login`.
- Si el backend responde correctamente, mira los roles.
- Si el usuario tiene rol `admin`, lo manda a `/dashboard-admin`.
- Si no, lo manda a `/home`.
- Si falla, muestra alerta de credenciales invalidas.

11. Servicio de autenticacion

Archivo:

```
src/app/core/services/auth.service.ts
```

URL base:

```
private apiUrl = `${environment.apiUrl}/api`;
```

Peticion de login:


```
login(email: string, password: string): Observable<any> {  
  return this.http.post(`${this.apiUrl}/login`, { email, password }).pipe(  
    tap((response: any) => {  
      localStorage.setItem('access_token', response.accessToken);  
      localStorage.setItem('user_name', response.name);  
      localStorage.setItem('user_email', response.email);  
      localStorage.setItem('user_roles', JSON.stringify(response.roles));  
    })  
  );  
}
```

Para que sirve:

- Envuelve la llamada HTTP al backend.
- Guarda el token de acceso.
- Guarda nombre, correo y roles del usuario.
- Permite consultar si hay token.
- Permite cerrar sesion eliminando datos de `localStorage`.

Metodos importantes:

```
getToken(): string | null  
isAuthenticated(): boolean  
logout(): void
```

12. Interceptor de autenticacion

Archivo:

```
src/app/core/services/auth.interceptor.ts
```

Codigo:

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {  
  const token = localStorage.getItem('access_token');  
  
  if (token) {  
    const cloned = req.clone({  
      setHeaders: {  
        Authorization: `Bearer ${token}`  
      }  
    });  
  }  
  
  return next(cloned);  
}
```

```
return next(req);  
}
```

Para que sirve:

- Revisa si existe `access_token` en `localStorage`.
- Si existe, clona la petición HTTP.
- Agrega el header:

```
Authorization: Bearer token
```

- Luego envía la petición al backend.

Esto evita tener que agregar manualmente el token en cada servicio.

13. Recuperación de contraseña

Ruta principal:

```
/auth/forgot-password
```

Archivo contenedor:

```
src/app/features/public/auth/forgot-password/forgot-password.ts
```

Rutas hijas:

```
src/app/features/public/auth/forgot-password/forgot-password.routes.ts
```

Código:

```
export const routes: Routes = [  
  {path: "email", component: Email},  
  {path: "code", component: Code},  
  {path: "reset", component: Reset},  
  {path: "", redirectTo: "email", pathMatch: "full"}  
]
```

Pasos:

1. `email`: el usuario ingresa correo.

2. **code**: el usuario ingresa codigo de verificacion.
3. **reset**: el usuario escribe nueva contrasena.

Estado actual:

- El flujo esta separado por pasos.
- Usa rutas hijas.
- No se evidencia todavia conexion completa con backend para enviar codigo y restaurar contrasena.

14. Dashboard de administrador

Ruta:

```
/dashboard-admin
```

Archivos:

```
src/app/features/dashboard/dashboard-admin/dashboard-admin.ts  
src/app/features/dashboard/dashboard-admin/dashboard-admin.html
```

Para que sirve:

- Mostrar panel principal del administrador.
- Mostrar metricas generales.
- Mostrar alertas recientes.
- Mostrar sidebar de administrador.
- Abrir modales de comentarios, informacion y actualizacion de datos.

Componentes usados:

- **NavbarDashboard**
- **SidebarAdmin**
- **Comments**
- **UpdateInformation**
- **RecordInformation**
- **RouterModule**
- **TranslateModule**

Metricas visuales actuales:

- Estudiantes.
- Rutas activas.
- Alertas del dia.

Alertas visuales actuales:

- Bus fuera de ruta.
- Freno brusco.

Estado actual:

- Los datos del dashboard son de ejemplo.
- Hay modales funcionales visualmente.
- No se evidencia consumo real de metricas desde backend.

15. Dashboard de superadministrador

Ruta:

```
/dashboard-superadmin
```

Archivos:

```
src/app/features/dashboard/dashboard-superadmin/dashboard-superadmin.ts  
src/app/features/dashboard/dashboard-superadmin/dashboard-superadmin.html
```

Para que sirve:

- Mostrar panel principal del superadministrador.
- Mostrar metricas de administradores, colegios y comentarios.
- Mostrar alertas.
- Mostrar sidebar de superadministrador.

Metricas visuales actuales:

- Administradores.
- Colegios registrados.
- Comentarios de hoy.

Estado actual:

- Los datos son visuales/de prueba.
- La vista esta lista para recibir datos reales cuando se conecten servicios.

16. Gestion de administrador

Rutas:

```
/admin/usuarios  
/admin/padres  
/admin/conductores  
/admin/familias  
/admin/buses  
/admin/paradas  
/admin/rutas
```

Estas rutas representan la gestion de:

- Estudiantes.
- Acudientes.
- Conductores.
- Familias.
- Buses.
- Paradas.
- Rutas.

La estructura se apoya en componentes reutilizables:

- **CardRegister**: formulario de registro.
- **CardList**: listado con buscador y acciones.
- **RecordInformation**: modal de detalles.
- **UpdateRecord**: modal de actualizacion.
- **DeleteRecord**: modal de eliminacion.
- **NavbarManage** o navbars de gestion.
- **SidebarAdmin**.

Estado actual:

- Las vistas estan armadas.
- Los listados usan datos mock.
- Los servicios **Users** y **RoutesBuses** existen, pero estan vacios.
- Las acciones de editar/eliminar muestran modales y hacen **console.log**, pero no llaman backend.

17. Gestion de superadministrador

Rutas:

```
/superadmin/admins  
/superadmin/schools
```

Archivos principales:

```
src/app/features/superadmin/admins/pages/admins/admins.ts  
src/app/features/superadmin/schools/pages/schools/schools.ts
```

Para que sirve:

- Gestionar administradores.
- Gestionar escuelas/colegios.

Componentes usados:

- **NavbarManage**

- `SidebarSuperadmin`
- `CardRegister`
- `CardList`
- `RecordInformation`
- `UpdateRecord`
- `DeleteRecord`

Ejemplo de uso en HTML:

```
<app-card-register type="admins"></app-card-register>
<app-card-list type="admins"
  (viewItem)="showDetails($event)"
  (editItem)="showUpdate($event)"
  (deleteItem)="showDelete($event)">
</app-card-list>
```

Para escuelas:

```
<app-card-register type="schools"></app-card-register>
<app-card-list type="schools"
  (viewItem)="showDetails($event)"
  (editItem)="showUpdate($event)"
  (deleteItem)="showDelete($event)">
</app-card-list>
```

Estado actual:

- Los formularios y listados son reutilizables.
- Los modales abren y cierran.
- Las acciones no consumen servicios reales todavia.
- Los servicios `Admins` y `Schools` existen, pero estan vacios.

18. CardRegister

Archivo:

```
src/app/shared/components/cards/card-register/card-register.ts
```

Para que sirve:

- Crear formularios reutilizables de registro.
- Cambiar los campos segun el tipo recibido por `@Input()`.

Tipos soportados:

```
export type RegisterType =  
  | 'estudiante'  
  | 'acudiente'  
  | 'conductor'  
  | 'familia'  
  | 'bus'  
  | 'parada'  
  | 'ruta'  
  | 'admins'  
  | 'schools';
```

Uso:

```
<app-card-register type="estudiante"></app-card-register>  
<app-card-register type="bus"></app-card-register>  
<app-card-register type="schools"></app-card-register>
```

Logica importante:

```
@Input() type: RegisterType = 'estudiante';  
formData: Record<string, any> = {};  
groupedFields: any[] = [];
```

Al iniciar:

```
ngOnInit(): void {  
  this.groupedFields = this.buildGroupedFields();  
}
```

Al enviar:

```
onSubmit(): void {  
  console.log('Datos del formulario:', this.formData);  
  // Aqui iria el servicio de registro  
}
```

Estado actual:

- Arma formularios dinamicos.
- Guarda valores en `formData`.
- Todavia no registra datos en backend.

19. CardList

Archivo:

```
src/app/shared/components/cards/card-list/card-list.ts
```

Para que sirve:

- Mostrar listados reutilizables.
- Filtrar datos con buscador.
- Emitir eventos para ver, editar o eliminar.

Tipos soportados:

```
export type CardType =  
  | 'estudiante' | 'acudiente' | 'conductor' | 'familia'  
  | 'bus' | 'parada' | 'ruta' | 'admins' | 'schools';
```

Eventos:

```
@Output() viewItem = new EventEmitter<RecordData>();  
@Output() editItem = new EventEmitter<RecordData>();  
@Output() deleteItem = new EventEmitter<RecordData>();
```

Buscador:

```
get filteredItems(): any[] {  
  if (!this.searchText?.trim()) return this.items;  
  const term = this.searchText.toLowerCase().trim();  
  return this.items.filter(item =>  
    Object.values(item).some(value =>  
      value?.toString().toLowerCase().includes(term)  
    )  
  );  
}
```

Para que sirve ese código:

- Si no hay texto, muestra todos los registros.
- Si hay texto, convierte todo a minúscula.
- Busca coincidencias en cualquier campo del registro.

Estado actual:

- Funciona con datos mock (`MOCK_DATA`).

- Permite buscar dentro de los registros.
- Permite abrir modales mediante eventos.
- Todavía no obtiene datos desde backend.

20. Modales principales

Carpeta:

```
src/app/shared/components/modal
```

Modales importantes:

Modal	Para que sirve
<code>record-information</code>	Muestra detalles de un registro.
<code>update-record</code>	Permite editar datos de un registro.
<code>delete-record</code>	Pide confirmación antes de eliminar.
<code>comments</code>	Permite enviar un comentario o inconveniente.
<code>update-information</code>	Permite actualizar información de escuela.
<code>language</code>	Permite cambiar idioma.
<code>themes</code>	Permite cambiar tema visual.
<code>modal-container</code>	Contenedor visual para modales.
<code>confirmations</code>	Mensajes de confirmación.

Estado actual:

- Los modales funcionan como UI.
- Algunos emiten eventos al componente padre.
- Varios todavía no están conectados con servicios backend.

21. Perfil del administrador

Ruta:

```
/admin/informacion
```

Archivos:

```
src/app/features/profile/pages/information-admin/information-admin.ts  
src/app/features/profile/pages/information-admin/information-admin.html
```

Para que sirve:

- Mostrar informacion del administrador.
- Mostrar foto/avatar.
- Permitir cargar una imagen local.
- Abrir modal de idioma.
- Abrir modal de tema.
- Navegar a cambio de correo.
- Navegar a cambio de contrasena.
- Navegar a cambio de telefono.

Codigo para cargar imagen:

```
onFileSelected(event: any) {  
  const file = event.target.files[0];  
  
  if (file) {  
    const reader = new FileReader();  
  
    reader.onload = () => {  
      this.imageUrl = reader.result;  
    };  
  
    reader.readAsDataURL(file);  
  }  
}
```

Para que sirve:

- Toma el archivo seleccionado.
- Lo lee como Base64.
- Lo muestra en pantalla como imagen de perfil.
- No lo sube todavia a backend.

Navegaciones:

```
changeEmail() {  
  this.router.navigate(['admin/change-email/email']);  
}  
  
changePassword() {  
  this.router.navigate(['admin/change-password/email']);  
}  
  
changeContact() {  
  this.router.navigate(['admin/change-contact/telephone']);  
}
```

22. Cambiar correo, contraseña y telefono

Cambiar correo

Ruta principal:

```
/admin/change-email
```

Pasos:

```
{path: "email", component: Email},  
{path: "code-first", component: CodeFirst},  
{path: "reset", component: Reset},  
{path: "code-second", component: CodeSecond},  
{path: "", redirectTo: "email", pathMatch: "full"}
```

Funcion:

- Pedir correo.
- Verificar codigo enviado al correo actual.
- Ingresar nuevo correo.
- Verificar codigo enviado al nuevo correo.

Cambiar contraseña

Ruta principal:

```
/admin/change-password
```

Pasos:

```
{ path: "email", component: Email },  
{ path: "code", component: Code },  
{ path: "reset", component: Reset },  
{ path: "", redirectTo: "email", pathMatch: "full" }
```

Funcion:

- Pedir correo.
- Verificar codigo.
- Crear nueva contraseña.

Cambiar telefono

Ruta principal:

```
/admin/change-contact
```

Pasos:

```
{ path: "telephone", component: Telephone },
{ path: "code-first", component: CodeFirst },
{ path: "reset", component: Reset },
{ path: "code-second", component: CodeSecond },
{ path: "", redirectTo: "telephone", pathMatch: "full" }
```

Funcion:

- Pedir telefono actual o contacto.
- Verificar codigo.
- Ingresar nuevo telefono.
- Confirmar codigo final.

23. Validador de contraseñas

Archivo:

```
src/app/shared/validator/password-match.validator.ts
```

Codigo:

```
export const passwordMatch = (firstControl: string, secondControl: string):
ValidatorFn => {
  return (formGroup: AbstractControl): ValidationErrors | null => {
    const passwordControl = formGroup.get(firstControl);
    const confirmPasswordControl = formGroup.get(secondControl);

    return passwordControl?.value === confirmPasswordControl?.value ? null : {
passwordNoMatch: true }
  }
}
```

Para que sirve:

- Recibe el nombre de dos campos.
- Obtiene ambos controles del formulario.
- Compara sus valores.
- Si coinciden, devuelve `null`.

- Si no coinciden, devuelve `{ passwordNoMatch: true }`.

Esto se usa para validar que la nueva contraseña y su confirmación sean iguales.

24. Traducciones

Carpeta:

```
public/assets/i18n
```

Archivos:

```
es.json  
en.json  
fr.json  
pt.json
```

Uso en HTML:

```
{{ 'login.right.title' | translate }}
```

Uso en `app.config.ts`:

```
provideTranslateService({  
  loader: provideTranslateHttpLoader({  
    prefix: '/assets/i18n/',  
    suffix: '.json',  
  }),  
})
```

Para que sirve:

- Permite cambiar textos según idioma.
- Evita escribir textos fijos directamente en cada componente.
- Centraliza los textos en archivos JSON.

Modal de idioma:

```
src/app/shared/components/modal/language/language.ts
```

Código importante:

```
selectedLang = localStorage.getItem('lang') ?? 'es';

confirm() {
  localStorage.setItem('lang', this.selectedLang);
  this.translate.use(this.selectedLang);
  this.languageChanged.emit(this.selectedLang);
  this.close.emit();
}
```

Para que sirve:

- Guarda el idioma elegido.
- Cambia el idioma con `translate.use`.
- Emite evento de cambio.
- Cierra el modal.

Observacion:

- Algunos textos se ven con caracteres mal codificados al leerlos desde consola, por ejemplo `contraseÃ±a` o `cÃ³digo`.
- Conviene revisar que los JSON esten guardados correctamente en UTF-8.

25. Temas visuales

Archivo:

```
src/app/shared/components/modal/themes/themes.ts
```

Codigo:

```
currentTheme = localStorage.getItem('theme') ?? '';

setTheme(theme: string) {
  this.currentTheme = theme;
  document.body.className = theme;
  localStorage.setItem('theme', theme)
}
```

Para que sirve:

- Guarda el tema seleccionado.
- Aplica una clase CSS al `body`.
- Recuerda el tema usando `localStorage`.

Tambien existe:

```
src/app/core/services/theme-context.ts
```

Codigo:

```
export class ThemeContext {  
  applyTheme = signal(true);  
}
```

Esto prepara un estado reactivo con `signal`, aunque actualmente su uso parece limitado.

26. Navbars y sidebars

Carpeta:

```
src/app/shared/components/navbar
```

Componentes:

Componente	Para que sirve
<code>navbar</code>	Barra publica de la landing.
<code>nav-component</code>	Navbar usado en login.
<code>navbar-dashboard</code>	Navbar de dashboards.
<code>navbar-admin</code>	Navegacion administrativa.
<code>navbar-manage</code>	Barra para vistas de gestion.
<code>sidebar-admin</code>	Menu lateral del administrador.
<code>sidebar-superadmin</code>	Menu lateral del superadministrador.

Estos componentes ayudan a separar la navegacion segun el contexto:

- Visitante publico.
- Usuario en login.
- Administrador.
- Superadministrador.
- Pantallas de gestion.

27. Estructura de carpetas

Carpeta	Para que sirve
<code>src/app/features/public</code>	Paginas publicas: home, contacto, login, recuperar contrasena.

Carpeta	Para que sirve
<code>src/app/features/dashboard</code>	Dashboards de admin y superadmin.
<code>src/app/features/admin</code>	Gestion de usuarios, buses, rutas y paradas.
<code>src/app/features/superadmin</code>	Gestion de administradores y escuelas.
<code>src/app/features/profile</code>	Perfil y cambios de correo, contrasena y telefono.
<code>src/app/shared/components</code>	Componentes reutilizables: cards, navbars, modales, cambios.
<code>src/app/core/services</code>	Servicios globales como autenticacion y tema.
<code>src/app/core/interceptors</code>	Interceptores globales.
<code>src/app/core/guards</code>	Guardas de rutas.
<code>src/app/core/config</code>	Configuraciones compartidas.
<code>src/environments</code>	Variables de entorno.
<code>public/assets/i18n</code>	Traducciones JSON.
<code>public/assets</code>	Recursos publicos como logo.

28. Estado real de las funciones

Funcion	Estado actual	Observacion
Home publica	Implementada	Usa navbar, main y footer.
Contacto	Implementado	Muestra informacion de contacto.
Login	Conectado parcialmente	Llama backend y guarda token.
Interceptor JWT	Implementado	Agrega <code>Authorization: Bearer.</code>
Recuperar contrasena	Visual/enrutado	Falta confirmar conexion completa a backend.
Dashboard admin	Visual	Usa datos de ejemplo.
Dashboard superadmin	Visual	Usa datos de ejemplo.
Gestion estudiantes/acudientes/conductores	Visual	Usa componentes reutilizables y mock data.
Gestion buses/paradas/rutas	Visual	Usa componentes reutilizables y mock data.
Gestion admins/schools	Visual	Modales listos, servicios vacios.
Modales ver/editar/eliminar	Visuales	Emiten eventos, falta persistencia real.

Funcion	Estado actual	Observacion
Perfil admin	Visual	Permite cambiar imagen localmente.
Cambiar correo/contrasena/contacto	Visual/enrutado	Falta confirmar integracion real.
Traducciones	Implementadas	Revisar codificacion UTF-8.
Temas	Implementado visualmente	Guarda tema en <code>localStorage</code> .
Guards de rol	Pendiente	Archivo existe, pero no tiene logica.
Config navbar/dashboard	Pendiente	Archivos existen, pero estan vacios.

29. Pendientes tecnicos

Puntos importantes para mejorar la web:

1. Implementar servicios CRUD reales para usuarios, rutas, buses, paradas, escuelas y administradores.
2. Reemplazar `MOCK_DATA` por datos del backend.
3. Conectar `CardRegister` con servicios de registro.
4. Conectar `UpdateRecord` con servicios de actualizacion.
5. Conectar `DeleteRecord` con servicios de eliminacion.
6. Implementar guardas de rutas por rol.
7. Proteger rutas privadas si el usuario no tiene token.
8. Redirigir segun roles reales: admin, superadmin u otros.
9. Corregir textos mal codificados.
10. Agregar manejo de errores mas claro en login y CRUD.
11. Implementar logout real desde navbar o perfil.
12. Conectar dashboards con metricas reales.
13. Conectar alertas reales.
14. Subir imagen de perfil al backend o almacenamiento.
15. Revisar que los archivos vacios de config/guards/interceptors no queden como deuda confusa.

30. Como ejecutar o validar

Desde la carpeta web:

```
cd "C:\Users\ROJAS\Desktop\ADS0\Guardian Escolar\dvlp-front\dvlp-web"
npm install
npm start
```

Por defecto, Angular suele levantar en:

```
http://localhost:4200
```

Para compilar:

```
npm run build
```

Para pruebas:

```
npm test
```

Validaciones manuales recomendadas:

1. Abrir `/home`.
2. Revisar navbar, contenido principal y footer.
3. Abrir `/contact`.
4. Abrir `/auth/login`.
5. Probar validaciones de correo y contraseña.
6. Probar login con backend encendido.
7. Verificar que se guarde `access_token` en `localStorage`.
8. Revisar que las peticiones posteriores envíen `Authorization`.
9. Abrir `/dashboard-admin`.
10. Revisar métricas, alertas, sidebar y modales.
11. Abrir rutas de gestión admin.
12. Probar buscador de `CardList`.
13. Probar ver, editar y eliminar en listados.
14. Abrir `/superadmin/admins` y `/superadmin/schools`.
15. Abrir `/admin/informacion`.
16. Probar cambio de tema.
17. Probar cambio de idioma.
18. Revisar flujos de cambio de correo, contraseña y teléfono.

31. Repaso rápido de código importante

Login con backend

```
this.authService.login(email, password).subscribe({
  next: (res) => {
    const roles = res.roles;
    if (roles.includes('admin')){
      this.router.navigate(['/dashboard-admin'])
    } else {
      this.router.navigate(['/home'])
    }
  },
  error: () => alert('Credenciales invalidas')
})
```

Guardar token

```
localStorage.setItem('access_token', response.accessToken);
localStorage.setItem('user_name', response.name);
localStorage.setItem('user_email', response.email);
localStorage.setItem('user_roles', JSON.stringify(response.roles));
```

Agregar token a peticiones

```
setHeaders: {
  Authorization: `Bearer ${token}`
}
```

Ruta con hijos

```
{ path: 'auth/forgot-password', component: ForgotPassword, children:
forgotPasswordRoutes }
```

En el archivo real esta dentro de `path: 'auth':`

```
{
  path: 'auth',
  children: [
    { path: 'login', component: Login },
    { path: 'forgot-password', component: ForgotPassword, children:
forgotPasswordRoutes }
  ]
}
```

CardList con filtro

```
return this.items.filter(item =>
  Object.values(item).some(value =>
    value?.toString().toLowerCase().includes(term)
  )
);
```

Emitir evento al componente padre

```
showDetails(item: RecordData): void {
  this.viewItem.emit(item);
}
```

```
}
```

Recibir evento desde HTML

```
<app-card-list type="admins"  
  (viewItem)="showDetails($event)"  
  (editItem)="showUpdate($event)"  
  (deleteItem)="showDelete($event)">  
</app-card-list>
```

Cambiar idioma

```
localStorage.setItem('lang', this.selectedLang);  
this.translate.use(this.selectedLang);
```

Cambiar tema

```
document.body.className = theme;  
localStorage.setItem('theme', theme)
```

32. Checklist de repaso

Para estudiar este proyecto, conviene poder responder:

- Que es Angular y para que se usa en la web.
- Donde estan las dependencias.
- Que hace `app.config.ts`.
- Donde estan las rutas principales.
- Como funciona una ruta hija.
- Que hace el login.
- Donde se llama al backend.
- Donde se guarda el token.
- Como se agrega el token a las peticiones.
- Que son los dashboards.
- Que diferencia hay entre admin y superadmin.
- Para que sirve `CardRegister`.
- Para que sirve `CardList`.
- Como funcionan los modales.
- Que datos son reales y que datos son mock.
- Donde estan las traducciones.
- Como se cambia el idioma.
- Como se cambia el tema.

- Que servicios estan vacios.
- Que falta conectar con backend.

33. Resumen final

La app web de Guardian Escolar ya tiene una base amplia: landing publica, contacto, login, autenticacion con token, interceptor JWT, dashboards, gestion visual de registros, modales, perfil, cambio de datos, temas y traducciones.

Lo mas importante para entenderla es que la web esta organizada por funcionalidades (**features**) y por componentes reutilizables (**shared**). El login es la parte mas conectada al backend actualmente, mientras que los modulos administrativos y de gestion todavia trabajan principalmente con datos de prueba y modales visuales.

Para una version mas completa, el siguiente paso seria conectar todos los formularios, listados, dashboards y modales con servicios reales del backend.